Introduction à Git

Michaël Soulignac

ESILV Nantes

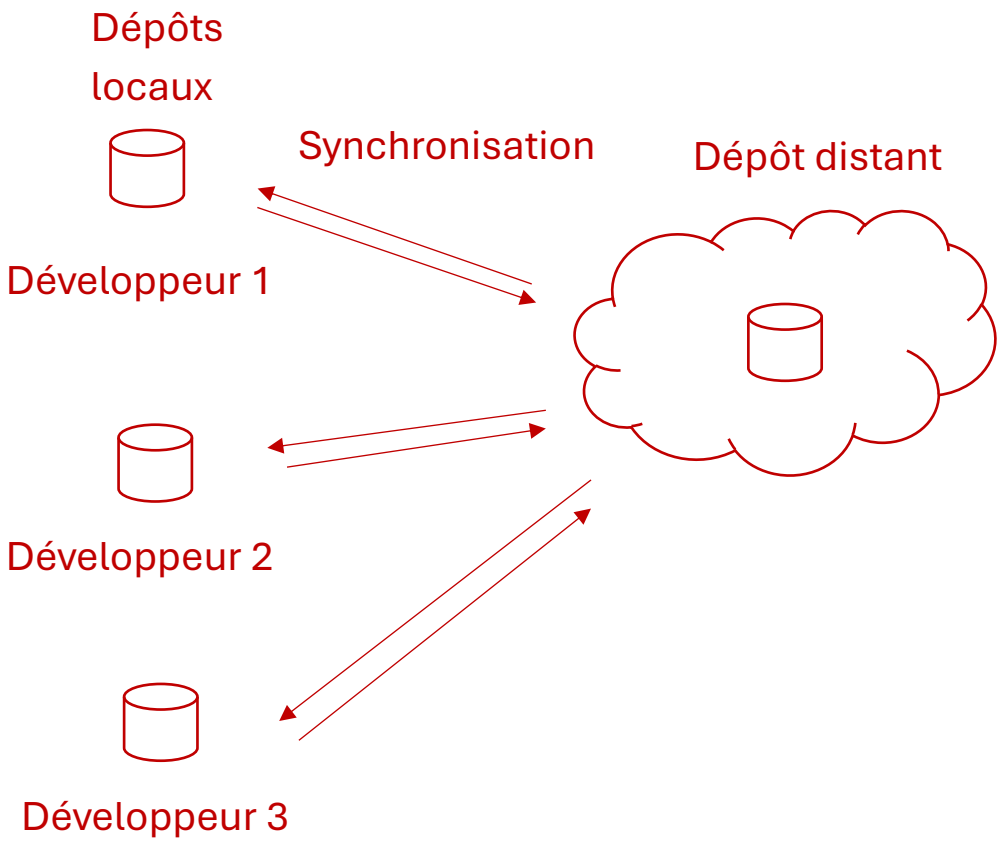
Concepts de base de Git

Qu'est que Git ?

- Système de gestion de versions
- Suivi des modifications
- Points de sauvegarde
- Travail collaboratif
- Incontournable en entreprise aujourd'hui
- Largement utilisé dans les projets *open source* (GitHub)

Architecture

- Dépôt distant :
  - Sur un serveur de l'entreprise ou d'un hébergeur
  - Version de référence
- Dépôts locaux:
  - Sur les postes de travail des développeurs
  - Copies locales du dépôt distant

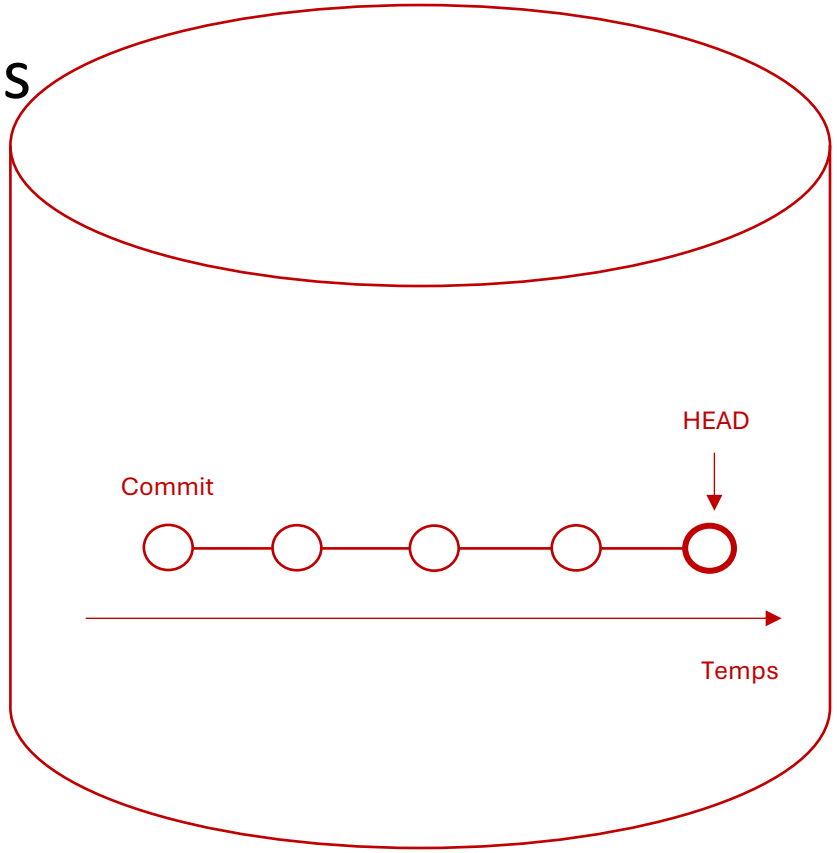


Contenu d'un dépôt

- *Commit* = validation de modifications

⇒ Point de sauvegarde

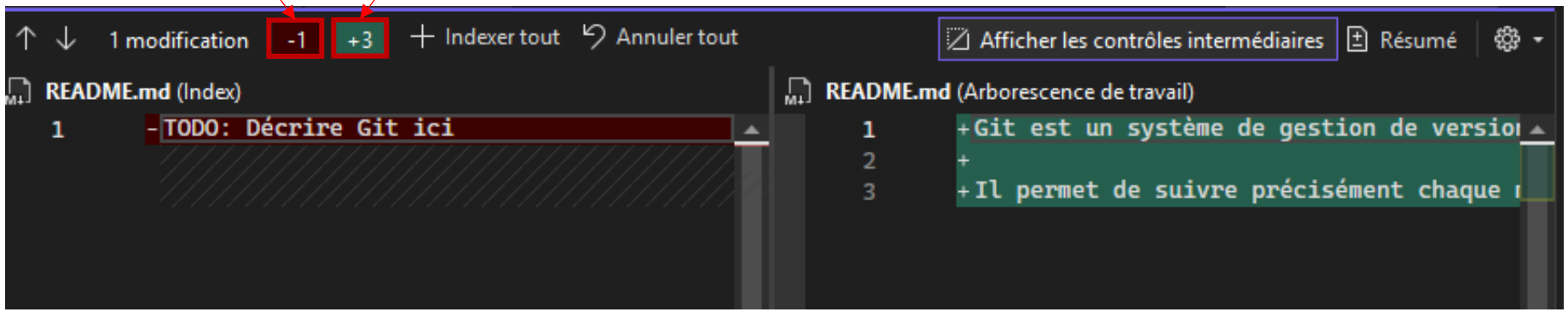
- Succession de commits = branche
- *HEAD* = version courante



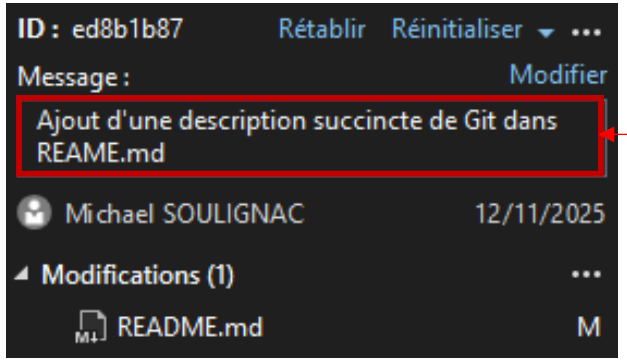
Contenu d'un *commit*

- Modifications (*diff*)

Supprimé Ajouté



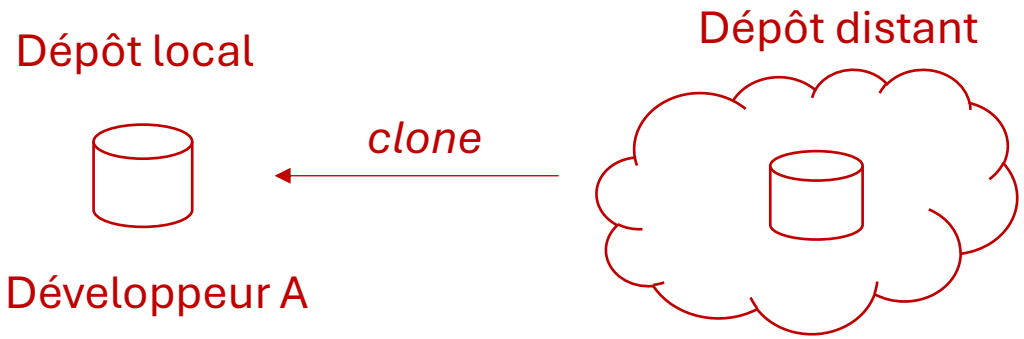
- Date, objet, auteur



Message de *commit* : doit **être clair, concis et informatif**

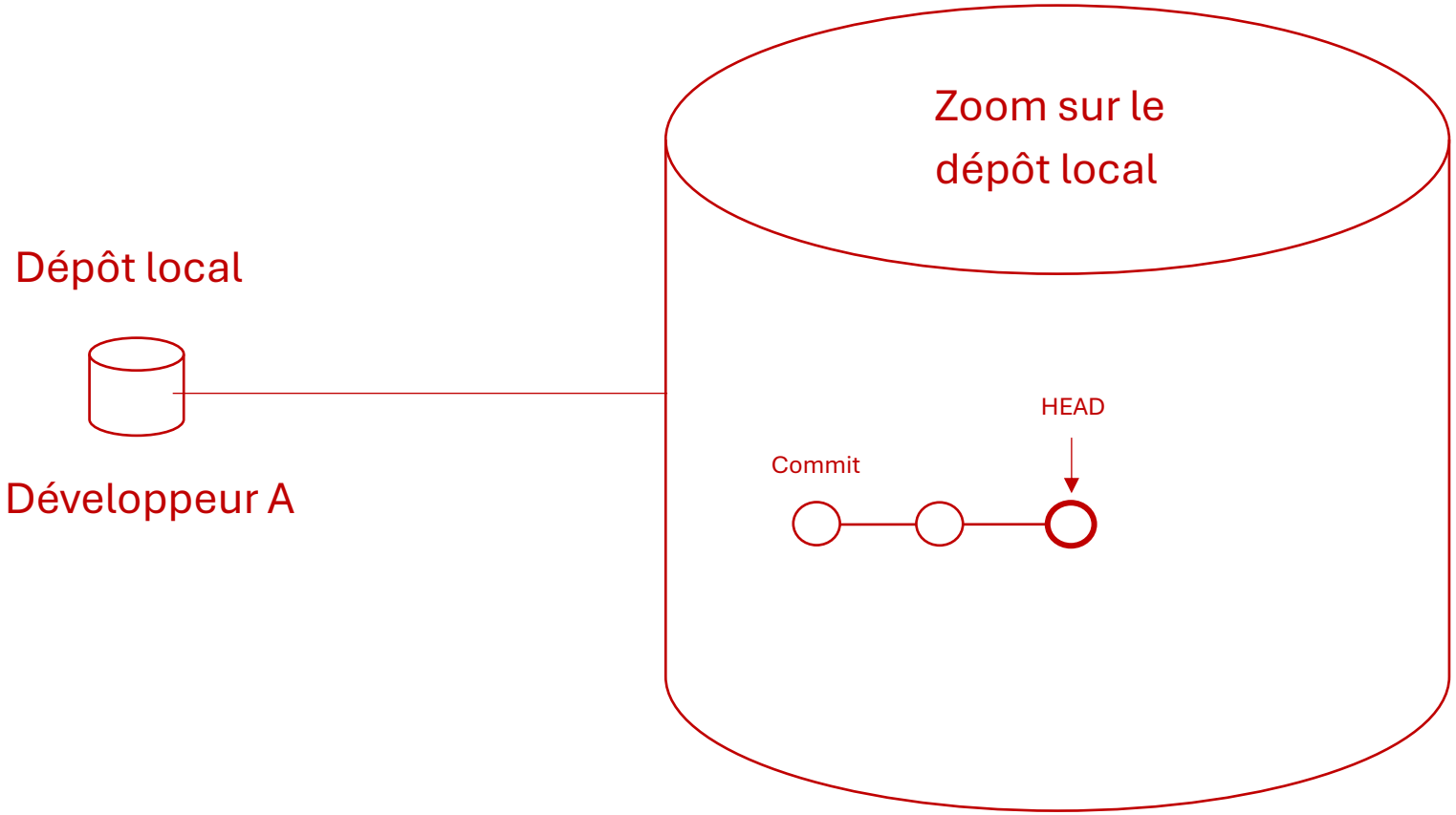
Flux de travail

1. Quand un développeur A rejoint un projet, il *clone* le dépôt distant



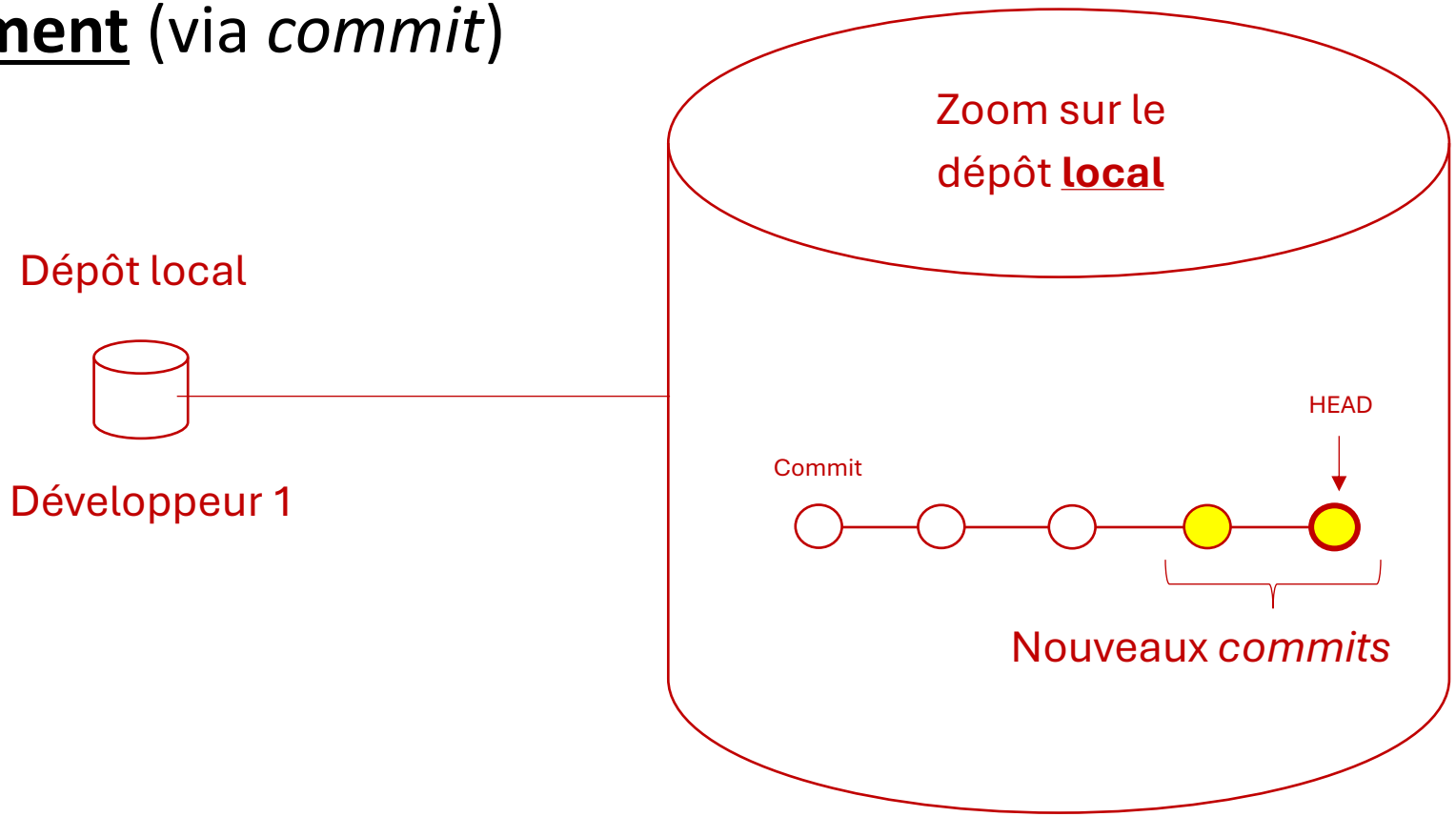
Flux de travail

1. Quand un développeur A rejoint un projet, il *clone* le dépôt distant



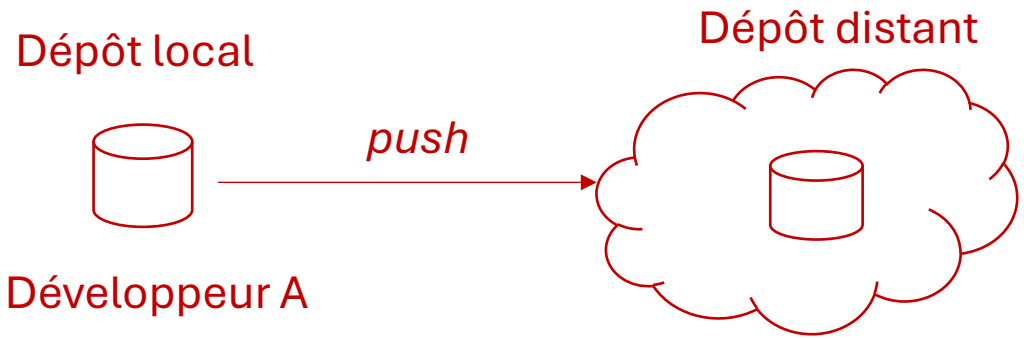
Flux de travail

2. Le développeur A effectue des modifications qu'il valide **localement** (via *commit*)



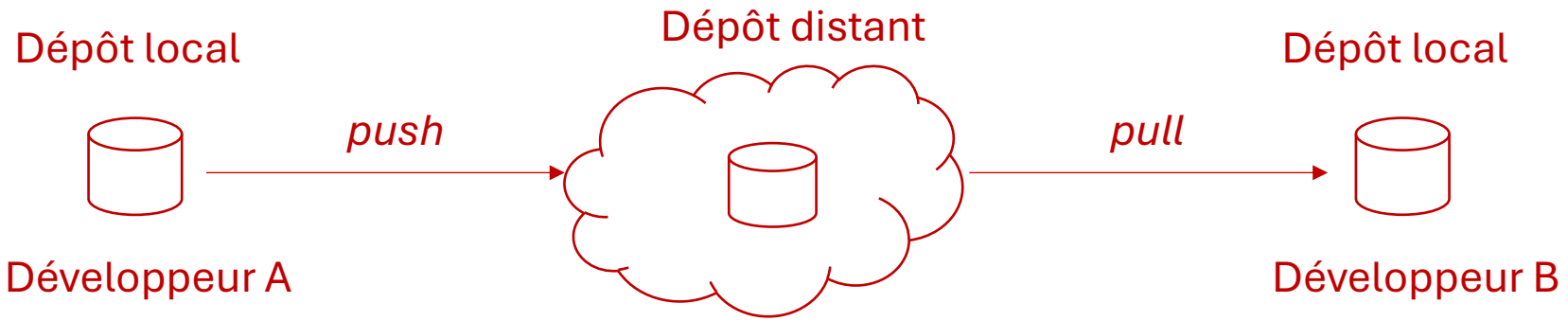
Flux de travail

3. Le développeur A envoie ses modifications sur le serveur pour les rendre visibles et utilisables par les autres (via *push*)



Flux de travail

4. Le développeur B récupère les modifications sur son poste (via *pull*).



GitHub

Qu'est-ce que GitHub ?

- GitHub est un hébergeur de dépôts Git

⇒ Il stocke les dépôts distants sur ses serveurs

- Mais c'est bien plus que cela : c'est une **forge logicielle**, permettant entre autres de :
  - Gérer des versions et des branches
  - Collaborer via des outils comme le suivi des bugs (*issues*) ou les demandes de fusion (*pull requests*)
  - Automatiser les tests et les déploiements

- C'est la plateforme de référence pour les projets *open source*.

Alternatives

Il existe des plateformes similaires, comme :

- GitLab : plateforme complète pour Git avec gestion de projets et intégration continue (CI/CD). Très utilisé en entreprise pour tout le cycle de développement.
- Bitbucket : développé par Atlassian, intégré à Jira et Confluence, souvent choisi par les équipes déjà dans l'écosystème Atlassian.
- Gitea : une solution open source légère pour héberger des dépôts Git, souvent utilisée par les petites équipes ou pour l'auto-hébergement.

Intégration dans Visual Studio

Interface en ligne de commande (*CLI*)

- A la base, Git se manipule dans un terminal avec une interface en ligne de commandes (*Command Line Interface* ou *CLI*)
- Par exemple, on peut effectuer un *commit* via :

```
git commit -m "message de commit"
```

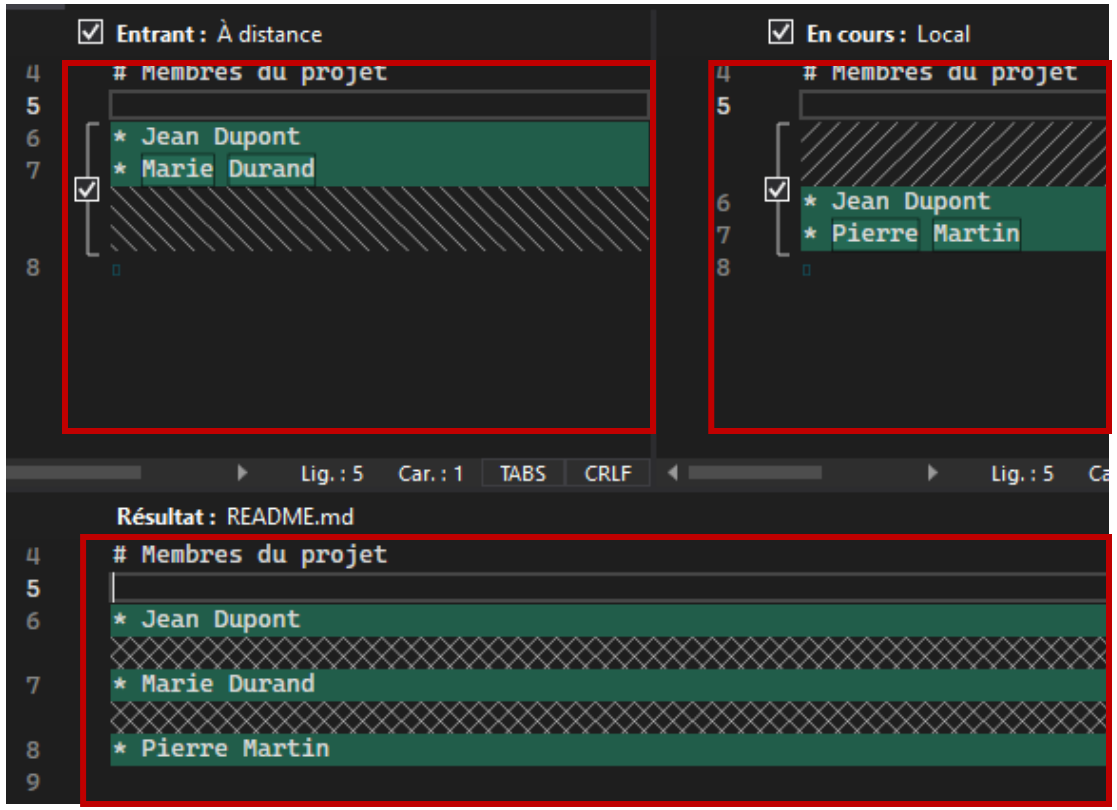
Interfaces graphiques (*GUI*)

- On peut manipuler Git via des interfaces graphiques comme:
  - Des outils dédiés : GitHub desktop, GitKraken, SourceTree…
  - Des intégrations dans les IDE, en particulier Visual Studio.
- Ces outils facilitent la visualisation (branches, *diff*) et des opérations parfois fastidieuses (résolution de conflit).
- En coulisse, ces outils effectuent des commandes Git.
  - Plus simple, mais plus opaque aussi.
  - Dans certaines situations complexes, il est parfois nécessaire de revenir à la ligne de commande.

Exemple: Résolution de conflit

Versions en conflit

Version locale



Version distante

Version fusionnée

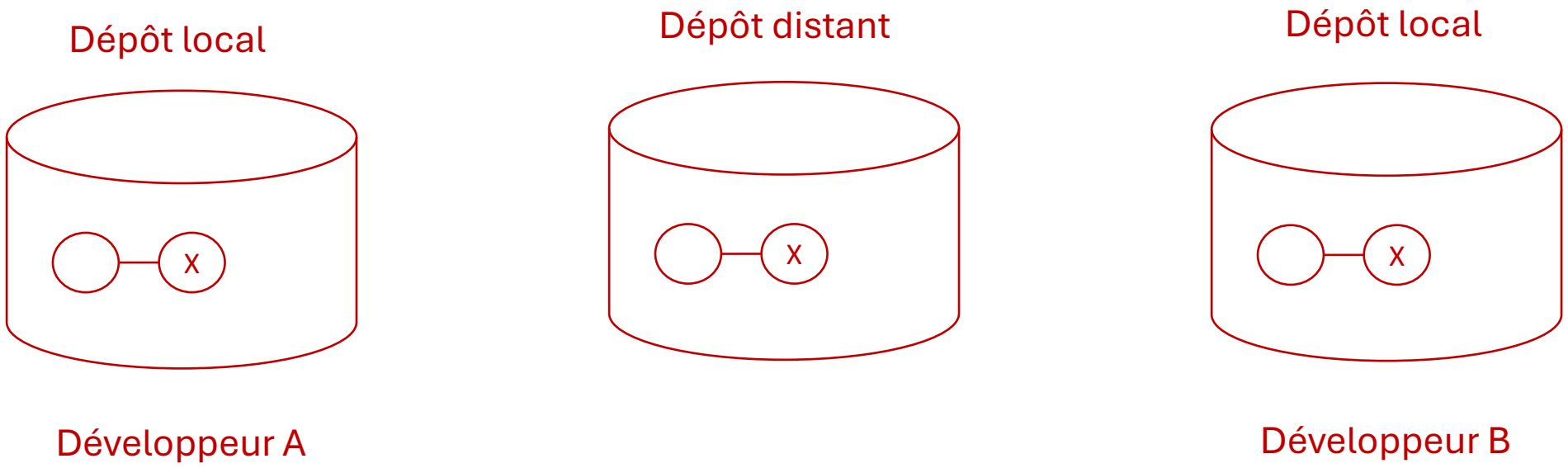
Annexe : Gestion des conflits

Qu’est-ce qu’un conflit ?

- Un **conflit** apparaît quand plusieurs développeurs modifient la même portion d’un fichier : Git ne sait pas quelle version conserver.
- C’est au développeur de le résoudre **manuellement**, par une opération appelée **fusion**, pendant laquelle il va :
  - Préciser quelles parties de code il souhaite récupérer dans chaque version
  - Faire les ajustements nécessaires (réagencement, adaptation…) pour que toutes ces parties fonctionnent ensemble.

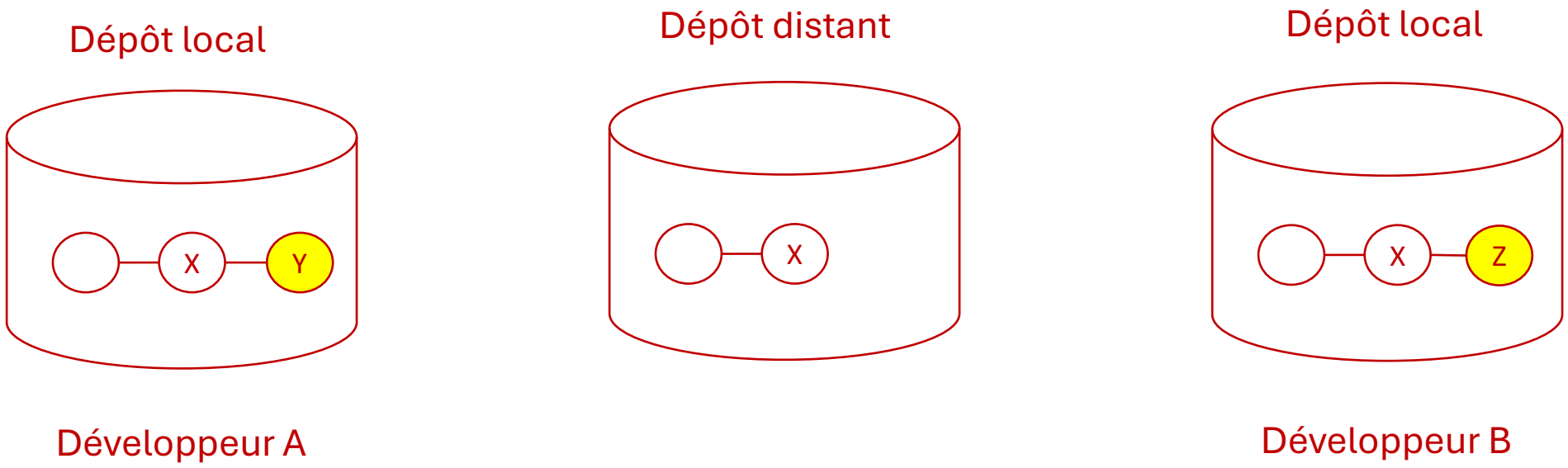
Comment naît un conflit ?

Supposons deux développeurs A et B avec des dépôts locaux à jour par rapport au dépôt distant (dernier *commit* : X)



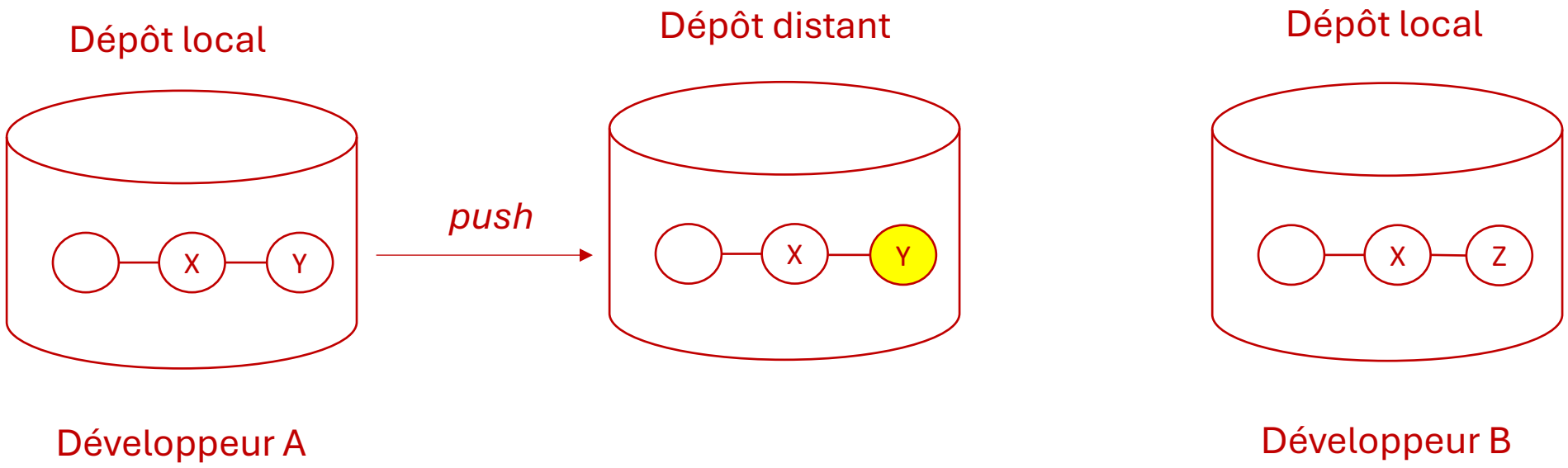
Comment naît un conflit ?

Les deux développeurs réalisent et valident localement des modifications



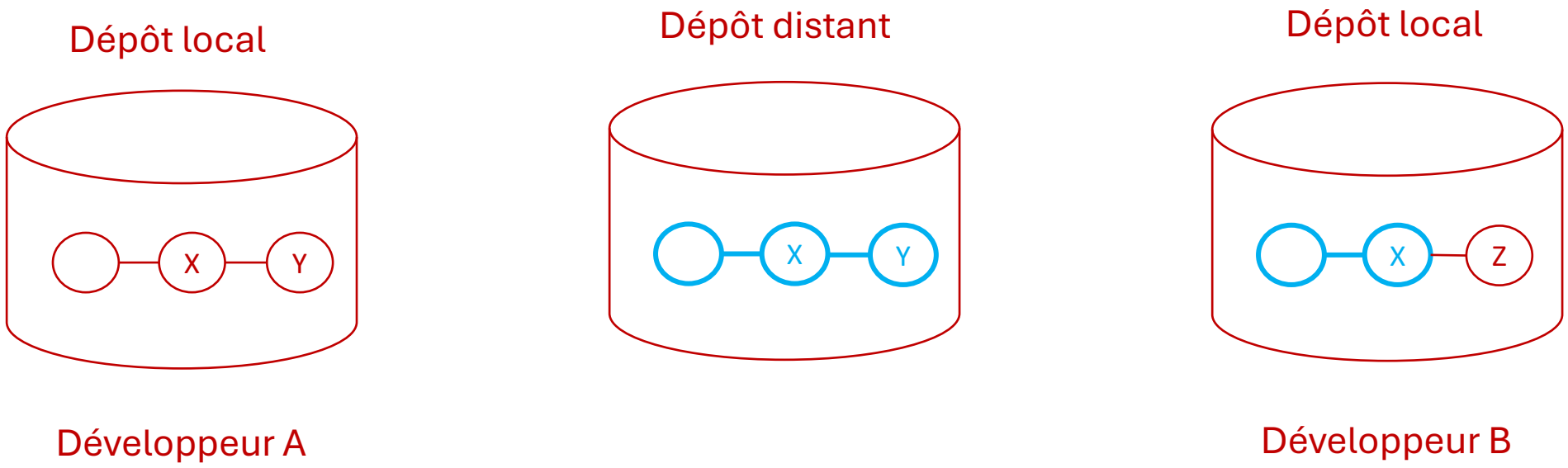
Comment naît un conflit ?

Le développeur A pousse ses modifications sur le serveur (sans problème)



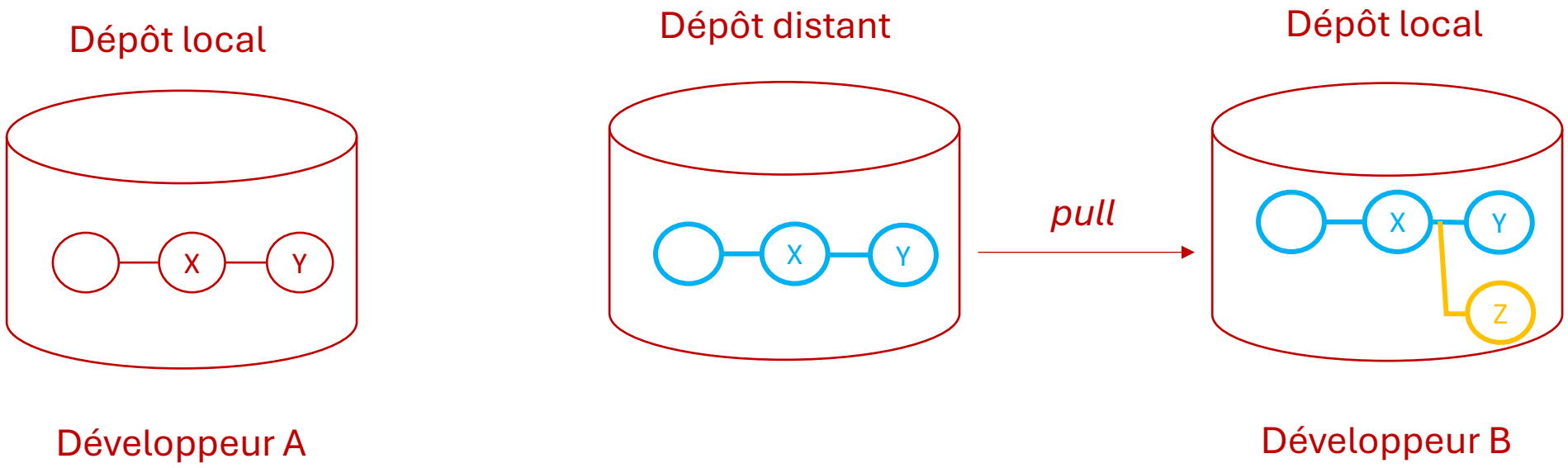
Comment naît un conflit ?

Le développeur B se retrouve en retard d'un *commit* par rapport au dépôt distant ⇒ Il ne peut **pas** pousser ses modifications



Comment naît un conflit ?

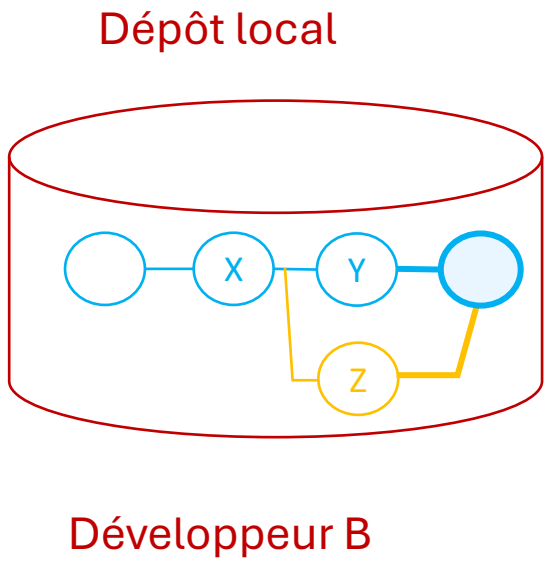
Le développeur B **doit** tirer les modifications (*pull*) pour être à jour ⇒ ceci provoque un embranchement



Comment naît un conflit ?

- Les nœuds Y et Z doivent être fusionnés.
- Il y a alors 2 scénarios possibles :

- Si les modifications effectuées dans Y et Z sont indépendantes ⇒ Git peut faire la fusion automatiquement
- Sinon, la fusion devra être faite manuellement: c'est le développeur B qui définira le contenu de version finale.



Conclusion

- Il n'y a jamais de conflit sur le dépôt distant.
- Un conflit apparaît localement, au moment d'un *pull*.
- Le développeur ayant réalisé le *pull* doit faire une fusion manuelle pour résoudre le conflit.
- Même si, techniquement, c'est ce développeur qui définit la version finale des fichiers, il ne doit pas le faire seul : ce choix doit être fait en concertation avec le reste de l'équipe.

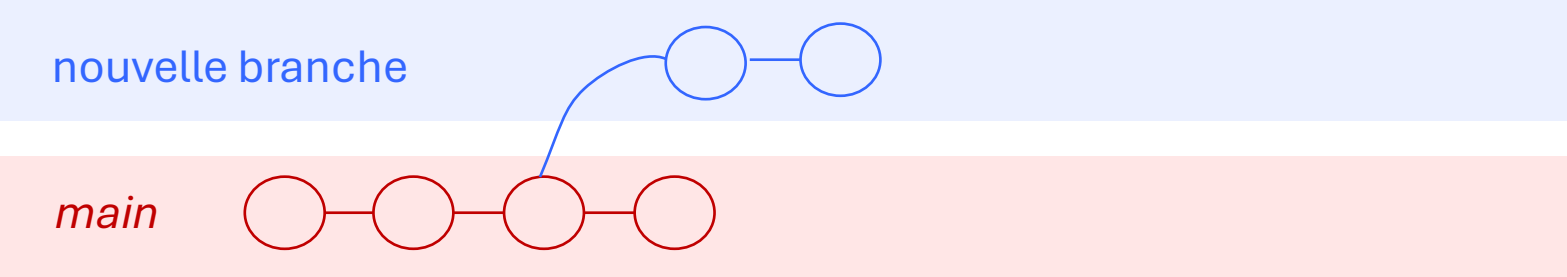
Annexe : Les branches (aperçu)

Création d'une branche

- Dans un projet réel, les développeurs ne travaillent pas sur la branche principale (*main*), qui est généralement verrouillée
- Pour chaque évolution du logiciel (ajout d'une fonctionnalité, correction d'un bug), on crée une branche dédiée

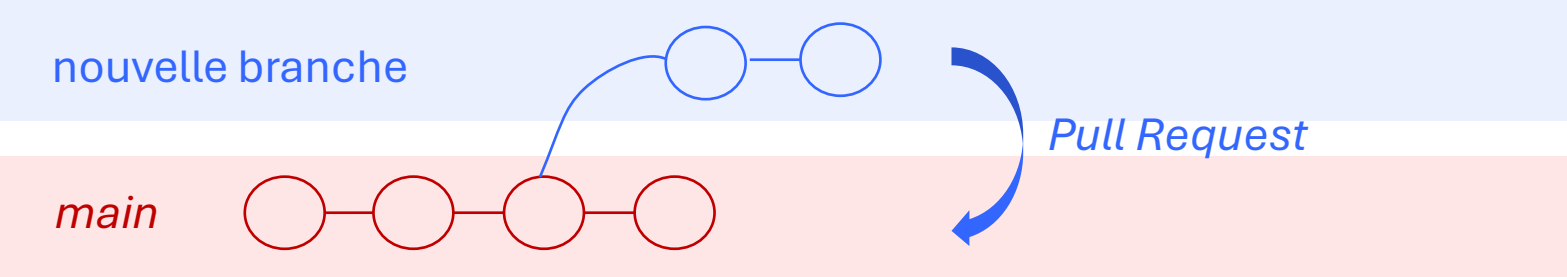
Travail sur la branche

- Les développeurs travaillant sur ces évolutions poussent des modifications sur cette branche
- La branche principale peut continuer à évoluer



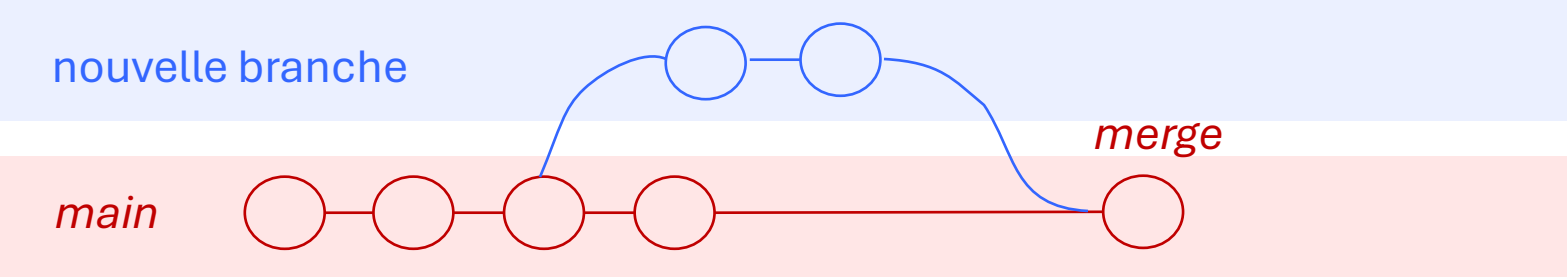
Création d'une *Pull Request*

- Les développeurs travaillant sur ces évolutions poussent des modifications sur cette branche
- Quand le travail est terminé, ils demandent de fusionner leur branche dans *main*, via une *Pull Request*.



Traitement d'une *Pull Request*

- Lors d'une *Pull Request*, d'autres développeurs font une revue de code, d'éventuelles remarques ou demandes de modification
- Une fois toutes les demandes traitées, un des développeurs ayant les droits requis fusionne la branche dans main (*merge*).



Pull Requests dans GitHub

Code | Issues | **Pull requests** 1 | Actions | Projects | Security | Insights | Settings

Numéro de PR

écriture de PSI en majuscules #2

Branche à fusionner

Edit | Code

Open [redacted] wants to merge 1 commit into `main` from `correction-typo`

Branche destination

Conversation 0 | Commits 1 | Checks 0 | Files changed 1 | +1 −1

[redacted] commented 17 minutes ago

Remplacement de psi par PSI

écriture de PSI en majuscules | Verified | 01abad5

Reviewers [redacted]

Réalisent des revues de code

Still in progress? Convert to draft

Assignees [redacted]

Suivent et valident la PR